Designed by Remus Teodorescu 2026

THE ARC

Why This Is the Important One

AND WHAT WOULD HAPPEN IF I SKIPPED IT

Most introductions to PyTorch present autograd as the thing that trains a neural network. That is true and it is not the useful framing, because it leaves you thinking of differentiation as something that happens inside a library you will never open.

The framing I want is different. Autograd is a general differentiation engine. Give it any computation built out of torch operations, tell it what to differentiate and with respect to what, and it returns exact derivatives — to machine precision, not approximated.

The distinction is not academic. Every exercise in lectures seven to twelve differentiates a network's output with respect to its input, in order to evaluate a differential equation residual. If you leave today thinking autograd is for weights, that whole idea will look like a trick when it arrives.

So this half-lecture spends ten slides on differentiating functions that are not networks at all, and only then builds the training loop. The order is deliberate.

TODAY, IN THREE PARTS

tensors	arrays, dtype, device, shapes
autograd	the engine — ten slides
the loop	forward, loss, zero, backward, step
and one warning	why tanh, not ReLU, from L7

Ten of twenty-six slides on differentiation, and none of them needs a network.

WHAT PART 2 SILENTLY ASSUMES

That you can differentiate a network's OUTPUT with respect to its INPUT and know why you would want to. It is assumed from L7.1 onwards and taught here.

An Array That Remembers What Happened to It

ALMOST EVERYTHING TRANSFERS DIRECTLY FROM NUMPY

A tensor has a shape, a dtype and elements, it indexes and slices exactly as a NumPy array does, and it broadcasts by exactly the rule you learned last lecture. If you can use NumPy you can use torch, and the translation table on the right is most of what you need.

One thing is added. A tensor can record the operations performed on it, building a graph of what was computed from what. That graph is what makes exact differentiation possible, and it is the entire difference between the two libraries as far as this course is concerned.

The graph is not a data structure you build. It appears as a side effect of doing arithmetic, it is thrown away when you differentiate, and for the first ten slides that is all you need to know about it.

One habit carries straight over, and it matters more here rather than less. Print the shape. A network changes shapes at every layer, and a wrong shape in torch fails in exactly the silent way L1.2 warned about.

THE TRANSLATION

np.array	torch.tensor
np.zeros, np.linspace	torch.zeros, torch.linspace
a @ b	a @ b — unchanged
a.reshape(-1, 1)	a.reshape(-1, 1) — unchanged
a.sum(axis=0)	a.sum(dim=0) — axis is dim
float64 by default	float32 by default
—	requires_grad, and the graph

Two rows differ. The last one is why this library exists for us.

dtype, device, and Why Neither Bites Yet

TWO ATTRIBUTES THAT CAUSE MOST FIRST-WEEK ERRORS

PyTorch's default floating type is float32, where NumPy's is float64. That is a deliberate choice — half the memory and, on a GPU, several times the speed — and it is why a NumPy array converted without a dtype argument produces a mismatch.

It also sets your tolerances. A first derivative computed in float32 agrees with a hand-derived one to about one part in ten thousand, not one in ten to the twelve. That is arithmetic, not a bug, and the Ex02 notebooks state the tolerance and its reason every time.

Device is where the tensor's memory lives: the CPU, or a particular GPU. Operations require all their tensors on the same device, and the error when they are not is clear.

Nothing in Part 1 needs a GPU. The largest network you will train here has about two thousand parameters and runs on a laptop in seconds, so device is fixed to the CPU throughout Ex02 and mentioned only so that the word is not new later.

DTYPE AT THE BOUNDARY

```
a = np.linspace(0, 1, 5)   # float64

x = torch.tensor(a, dtype=torch.float32)
x.dtype                    torch.float32

# say it. every time.
```

TOLERANCES THAT FOLLOW FROM IT

1e-4	a first derivative, float32
1e-3	a second derivative
5e-3	against a finite difference
float64	when precision is the question

The (n, 1) Column, and What a Layer Is

ONE CONVENTION, AND ONE MATRIX MULTIPLICATION

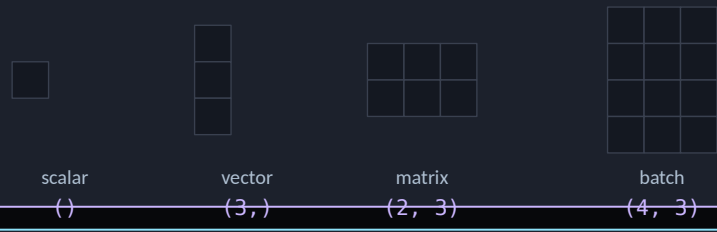
Every network input in this course is a column: shape n by one, not shape n. That is not arbitrary — a linear layer expects a batch of samples with one row each, and a bare one-dimensional tensor will either fail or broadcast into something you did not want.

So the first line of every exercise turns a grid into a column with reshape minus one, one. Ex02's core module calls it as_input, and the Part 2 core libraries call it the same thing, deliberately.

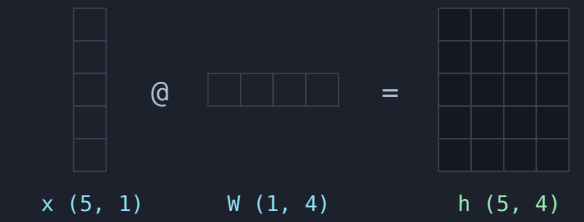
A linear layer is a matrix multiplication and an addition. Inputs times a weight matrix, plus a bias vector, broadcast across the batch. That is the whole of it, and writing it by hand once — which Ex02 notebook one asks you to do — is worth more than reading the documentation for nn.Linear.

The parameter count follows: input features times output features, plus output features. A layer from one input to twenty units has forty parameters, and you should be able to do that arithmetic in your head by lecture four.

FOUR SHAPES, AND WHAT .shape PRINTS



A BATCH IS A COLUMN OF ROWS



A LAYER, BY HAND

```
x = t.reshape(-1, 1)      # (n, 1)
h = x @ W + b             # (n, 4)
h = torch.tanh(h)        # 1*4 + 4 = 8 parameters
```

Mark the Tensor You Care About

ONE ARGUMENT, AND WHAT IT SWITCHES ON

requires_grad equals True marks a tensor as one that autograd should track. From that point, every operation involving it records what it did.

Note what the argument is attached to. Not a model, not a session — a tensor. You are saying this particular array is a quantity I may want to differentiate with respect to, and torch is free to ignore everything else.

That is why the framing on slide two matters. There is nothing here about networks. x is a grid of positions and it is tracked because I said so.

Two properties worth printing while you learn. requires_grad, which says whether it is tracked, and is_leaf, which says whether it is an input to the graph rather than something computed inside it. A tensor you created and marked is a leaf; anything computed from it is not.

THE MARKED TENSOR

```
x = torch.tensor(2.0, requires_grad=True)

x.requires_grad          True
x.is_leaf                True
x.grad_fn                None
```

WHAT IT IS NOT ATTACHED TO

not a model	a single tensor, by name
not a session	no global mode is set
not the weights	x here is a position

The Graph Builds Itself

AS A SIDE EFFECT OF COMPUTING THE ANSWER

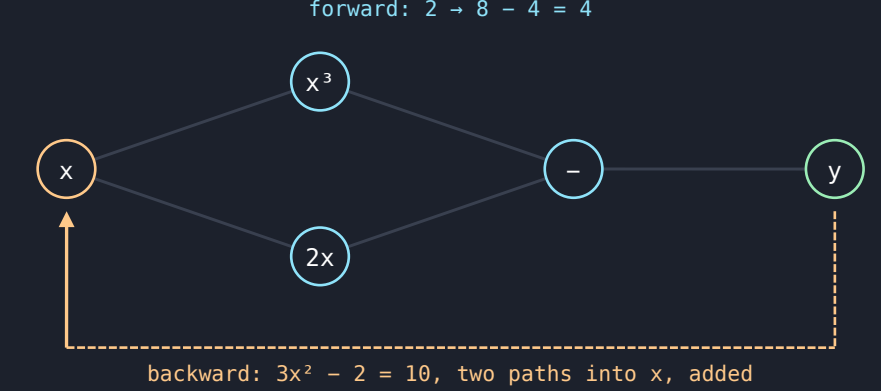
Now compute something with the marked tensor. y equals x cubed minus two x. Torch evaluates it and gets four, exactly as NumPy would — and at the same time it records that y came from a subtraction, whose left operand came from a power, whose base was x.

That record is the graph on the right. It is built forwards, while you compute, and it is not something you construct or ask for.

The evidence that it exists is grad_fn. x has none, because it is a leaf. y has one, and it names the last operation that produced it. Printing grad_fn is the quickest way to check whether a tensor is still connected to the thing you want to differentiate.

One consequence worth noticing now: the graph is built from the operations you actually performed. There is no expression to parse and no formula to simplify, which is why autograd works on code containing loops and conditionals.

BUILT FORWARDS, WHILE YOU COMPUTE



Amber is the leaf. Green is what you computed. Dashed is the way back.

THE EVIDENCE

```
y = x ** 3 - 2 * x
y                   tensor(4., grad_fn=<SubBackward0>)
x.grad_fn           None
```

Ask for the Derivative

ONE CALL, AND AN ANSWER YOU CAN CHECK IN YOUR HEAD

torch.autograd.grad takes what you want to differentiate and what you want to differentiate it with respect to. It walks the graph backwards, applying the chain rule at each node, and returns a tuple of gradients — hence the comma on the left of the assignment.

The example is deliberately trivial so that you can check it. y is x cubed minus two x, so the derivative is three x squared minus two. At x equals two that is ten. Autograd returns ten point zero.

Check every derivative you compute this way at least once against one you wrote by hand. Not because autograd is unreliable — it is not — but because the thing that goes wrong is your function, and a checked derivative at one point catches a sign error that a plausible-looking curve will not.

Note also that the answer is exact. Not accurate to some tolerance you chose: exact to the precision of the arithmetic, because it is the chain rule applied to the operations you actually performed.

$$y = x^3 - 2x, \qquad \frac{dy}{dx} = 3x^2 - 2$$

THE CALL, AND THE CHECK

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 - 2 * x

dy_dx, = torch.autograd.grad(y, x)

dy_dx                 tensor(10.)

# by hand: 3*x**2 - 2 = 3*4 - 2 = 10
hand = 3 * 2.0**2 - 2
abs(dy_dx.item() - hand)   0.0
```

The trailing comma is not a typo. grad returns a tuple, one entry per input you asked about.

What It Is, and Two Things It Is Not

THE DISTINCTION IS WORTH ONE SLIDE AND SAVES CONFUSION LATER

NOT SYMBOLIC DIFFERENTIATION

It does not produce a formula. There is no expression to look at afterwards and no simplification step. It applies the chain rule numerically to the operations that actually ran — which is why it copes with loops, conditionals and anything else your code does.

NOT FINITE DIFFERENCES

There is no step size and no approximation. The answer is exact to the precision of the arithmetic, and it costs one backward pass regardless of how many inputs you differentiate with respect to. Slide 14 measures the difference.

IT IS THE CHAIN RULE, PLUS BOOKKEEPING

Each operation knows its own derivative. The graph records which operations ran and in what order. Differentiating is walking that record backwards and multiplying. There is nothing else in it.

AND THE PRACTICAL CONSEQUENCES OF EACH

Because it is not symbolic, you cannot print the derivative as an equation, and a mistake in your function is invisible until you check a value — which is why slide eight insisted on the hand check. Because it is not a finite difference, there is no step size to tune and no accuracy trade-off to worry about, which removes an entire class of decision that dominates conventional numerical work.

And because it is bookkeeping over operations that actually ran, everything must be written in torch. A NumPy call in the middle of your function silently breaks the chain: the numbers come out right and the derivative comes out wrong, or missing. That is slide sixteen.

A Vector Output Needs One More Argument

grad_outputs, AND WHY A VECTOR OF ONES IS RIGHT HERE

The first example differentiated a scalar. On a grid, u is a vector of a hundred and one values, and the derivative of a vector with respect to a vector is a Jacobian — a hundred and one by a hundred and one numbers, almost all of them zero.

Autograd will not build that. What it computes is a vector–Jacobian product: you supply a vector, and it returns that vector multiplied by the Jacobian. The argument is grad_outputs.

For an elementwise map — where u_i depends only on x_i — the Jacobian is diagonal, so multiplying by a vector of ones gives exactly the elementwise derivatives. That is why ones is right here, and it is worth understanding rather than copying: it is right because the map is elementwise, not because it is conventional.

This is the argument people copy from a tutorial without reading. Reading it once now means that when you meet a case where ones is not right — a loss that already sums, for instance — you will notice.

$$u(x) = e^{-x}\sin(3x)$$

THE CALL, ON A GRID

```
x = torch.linspace(0, 2, 101,
                   requires_grad=True)
u = torch.exp(-x) * torch.sin(3 * x)

du, = torch.autograd.grad(
    u, x, torch.ones_like(u))
du.shape              torch.Size([101])
```

AND CHECKED AGAINST THE HAND ANSWER

$$\frac{du}{dx} = e^{-x}\big(3\cos(3x) - \sin(3x)\big)$$

101 points, max absolute error about 1e-6 in float32.

Second Derivatives, and create_graph

ONE ARGUMENT, WITHOUT WHICH PART 2 DOES NOT WORK AT ALL

The first derivative is itself a computation. If it was recorded, you can differentiate it again — and a second-order differential equation needs exactly that.

By default the graph is discarded when you differentiate, because keeping it costs memory and most training runs never need it. create_graph equals True tells torch to keep it, so that the first derivative comes back as a tracked tensor rather than as plain numbers.

Forgetting it has one precise symptom, worth memorising because you will meet it: the first derivative comes back with requires_grad False and grad_fn None, and the second call raises with the message that element nought of tensors does not require grad.

So in this course, every first derivative you intend to differentiate again is taken with create_graph equals True. Ex02's core module wraps that into a function called grad, and the Part 2 core libraries define the same function with the same name — so the line you write in week two is the line you read in week seven.

TWICE, PROPERLY

```
def grad(y, x):
    return torch.autograd.grad(
        y, x, torch.ones_like(y),
        create_graph=True)[0]

u_x  = grad(u, x)
u_xx = grad(u_x, x)

# without create_graph:
# RuntimeError: element 0 of tensors
# does not require grad
```

grad and d2 are named identically in Ex02 and in every Part 2 core library. That is deliberate.

THE POINT OF THE LECTURE

With Respect to the Input

Autograd differentiates with respect to anything you marked — including a network's INPUT, not only its parameters. That one fact is the mechanism of every exercise in lectures seven to twelve.

THE SAME CALL, WITH A NETWORK IN IT

Everything so far differentiated an analytic function. Replace that function with a neural network and nothing about the call changes. Mark the input, evaluate the network, ask for the gradient of the output with respect to the input.

What you get back is du by dx where u is whatever the network currently computes. It is a function of the network's parameters, so it changes as training proceeds — and, crucially, it is itself differentiable, which is what lets a loss be built out of it.

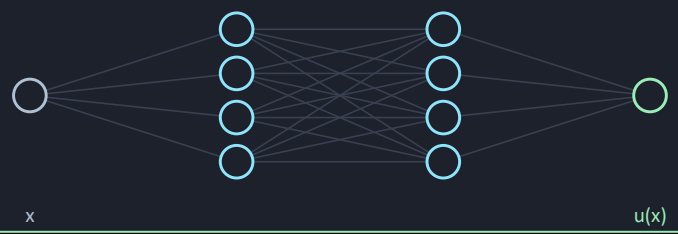
Check it, the first time, against a central difference of the network itself. Ex02 notebook two does exactly that, and it is the moment the idea stops being abstract.

If you take one thing from this recording, take this slide.

A NETWORK, DIFFERENTIATED IN x

```
x = as_input(torch.linspace(0, 2, 101))
x.requires_grad_(True)

u = net(x)              # (101, 1)
u_x = grad(u, x)        # du/dx
u_xx = grad(u_x, x)     # d2u/dx2
```



THE POINT OF THE LECTURE

A Residual Is a Loss with No Data In It

THE FIRST PHYSICS-INFORMED OBJECT IN THIS COURSE

Take a differential equation and move everything to one side. What is left is the residual: a function of position that is zero when the equation is satisfied and non-zero when it is not.

The equation on the right is the Helmholtz residual, and you can now evaluate it for any u, because you can differentiate u twice. Ex02 asks you to evaluate it on a function that solves the equation — you get something of order ten to the minus five, which is float32 noise — and then on one that does not, where you get something of order one.

Now look at what that gives you. A number that measures how wrong a candidate solution is, computed from the candidate alone, with no measured data anywhere in it.

That is the conceptual jump of the entire second course, and it costs one slide here. If a residual is a number, and it is differentiable with respect to the network's parameters, then it is a loss — and minimising it trains a network to satisfy a differential equation without ever being shown a solution.

MOVE EVERYTHING TO ONE SIDE

$$r(x) = \frac{d^2u}{dx^2} + k^2 u(x)$$

u solves it	r ≈ 1e-5 — float32 noise
u does not	r of order 1

AND THEREFORE A LOSS

```
u    = net(x)
u_xx = grad(grad(u, x), x)
r    = u_xx + k**2 * u
loss = (r ** 2).mean()

# no measured data anywhere
```

Autograd Against Finite Differences

THE ARGUMENT, MEASURED OVER ELEVEN DECADES OF STEP SIZE

The obvious way to differentiate numerically is a central difference. It is one line and it needs no library, so why not use it?

Three reasons, and the figure shows the first. It has a step size and there is no good choice for it. Make h too large and the truncation error dominates, falling as h squared. Make it too small and floating-point cancellation dominates, rising as machine epsilon over h. The two meet at a best case around ten to the minus eleven — and you had to know that to choose h.

Second, it costs two evaluations per derivative per dimension. In a physics-informed problem with three spatial dimensions and second derivatives that becomes expensive quickly, where autograd costs one backward pass regardless.

Third, and decisive: the loss you are minimising is built from these derivatives. A loss made of approximations has a noise floor, and the optimiser will happily spend its time chasing that noise rather than the physics.

$$\frac{u(x+h)-u(x-h)}{2h}=\frac{du}{dx}+O\left(h^{2}\right)$$
$$+O\left(\frac{\epsilon}{h}\right)$$

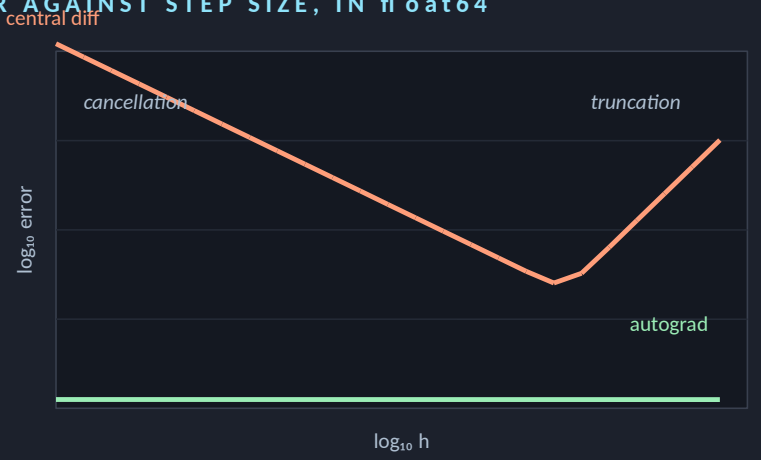
FINITE DIFFERENCE

Needs a step h. Error of order h squared. Two evaluations per input.

AUTOGRAD

No h at all. Exact to machine precision. One backward pass.

ERROR AGAINST STEP SIZE, IN float64



Best case near 1e-11, and only if you knew to look for it. Autograd is flat at machine epsilon and has no h at all.

BRIDGE TO PART 2

Why Every Network in L7–L12 Uses tanh

A CONSEQUENCE OF THE LAST FOUR SLIDES, NOT A STYLE CHOICE

A ReLU network is piecewise linear. Its first derivative is therefore piecewise constant, and its second derivative is exactly zero everywhere it exists.

Now put that together with slide thirteen. A second-order residual is built from a second derivative. If that second derivative is exactly zero, the residual carries no information about the network's parameters at all — the gradient of the loss with respect to every weight is zero, and training does nothing.

That is why every network in lectures seven to twelve uses tanh, whose second derivative is of order one. It is one line of code and it is the difference between a physics-informed network that trains and one that sits at its initial loss forever.

Ex02 notebook two demonstrates it directly: identical networks, identical seed, one with tanh and one with ReLU. The tanh second derivative is of order one; the ReLU second derivative is exactly zero, at every point. You will not need this today. Write it down anyway.

Three Ways to Lose the Graph

EACH IS USEFUL SOMEWHERE AND FATAL INSIDE A LOSS

.detach()

Returns a tensor with the same values and no history. Right for taking a snapshot to plot or to log; fatal if the result is then used in a loss, because the gradient path is gone and nothing complains.

.item() and .numpy()

Leave the tensor world entirely, producing a Python float or a NumPy array. Right at the end of a computation and at no other time. A NumPy call in the middle of your function silently breaks the chain.

with torch.no_grad():

Switches recording off for a block. Right for evaluation, where you want the numbers and not the derivatives, and it saves real memory. Wrong anywhere inside the training step.

THE ONE RULE THAT COVERS ALL THREE

Inside anything you intend to differentiate, use torch operations on tensors and nothing else. No NumPy, no float conversions, no detach. Convert at the very end, when you are plotting or printing.

The failure is quiet, which is why it gets a slide. A detached tensor still has the right numbers, so the loss still evaluates and still prints something plausible. What you notice is that the loss does not fall — and that is a hard symptom to diagnose from first principles.

THE DIAGNOSTIC, IN TWO LINES

```
print(t.requires_grad, t.grad_fn)

True  <MulBackward0>   connected
False None             detached
```

A loss that will not fall: check this first.

The Shape You Will See Forty Times

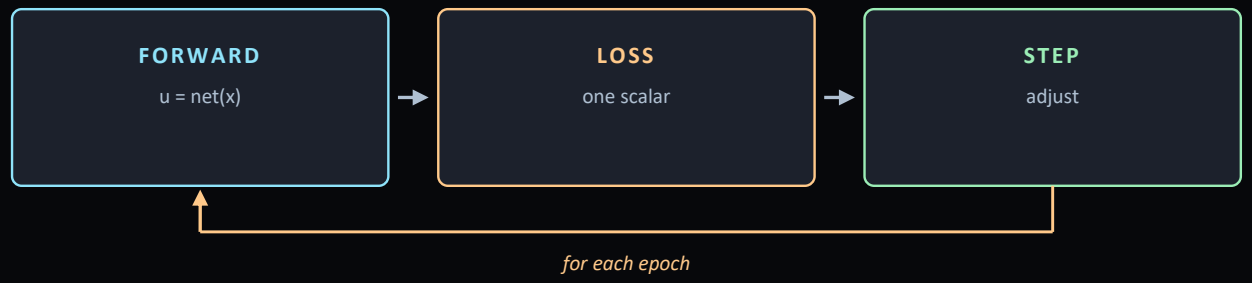
FIVE LINES, AND THE ORDER IS NOT NEGOTIABLE

Forward: run the model and build the graph. Loss: reduce the disagreement to one scalar. zero_grad: clear the gradients from the last step. backward: walk the graph and accumulate the gradients. step: update every parameter using them.

That is the whole training loop, and every training loop in this course — supervised or physics-informed, one layer or twelve — is that skeleton with a different loss in the second line.

The order matters in two specific places. zero_grad must come before backward, because backward adds to whatever is already there. And step must come after backward, because it uses what backward computed. Everything else about the loop is yours to arrange.

Note the direction of the last two arrows on the right. backward uses the same graph autograd has been using for the last ten slides — the only difference is that it accumulates into parameter dot grad rather than returning a value.



THE SKELETON

```
opt = torch.optim.Adam(net.parameters(),
                       lr=1e-3)

for epoch in range(2000):
    pred = net(t)              # 1 forward
    loss = ((pred - y)**2).mean()  # 2 loss
    opt.zero_grad()            # 3 clear
    loss.backward()            # 4 backward
    opt.step()                 # 5 update
```

$$\mathcal{L} = \frac{1}{N}\sum_{k=1}^{N}(\hat{y}_k - y_k)^2$$

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

Lines One and Two

THE FORWARD PASS, AND WHY THE LOSS MUST BE A SCALAR

Calling the model runs it and builds the graph. Nothing is differentiated yet — this is the same forward computation you have been doing since slide seven, and the graph appears the same way.

The loss must be a single number. Not a vector of errors per sample: one scalar. That is not a limitation of the library, it is what differentiation means here — a gradient is defined for a scalar function of the parameters, and a vector of losses has no single gradient.

So a loss always ends in a reduction. Mean, usually, because it makes the value independent of how many samples you used and therefore comparable between runs. Sum is defensible and makes your effective learning rate depend on the batch size, which is a trap.

For supervised learning the loss compares predictions with measurements. For a physics-informed network it is the mean squared residual from slide thirteen. Same line, same shape, and lecture six point one derives which loss belongs to which problem rather than leaving it to taste.

TWO LOSSES, SAME SHAPE

```
# supervised
pred = net(t)
loss = ((pred - y)**2).mean()

# physics-informed
r = u_xx + k**2 * u
loss = (r**2).mean()
```

WHY A SCALAR

A gradient is defined for a scalar function of the parameters. A vector of per-sample errors has no single gradient, so every loss ends in a reduction — and mean rather than sum, so the value is comparable between runs of different size.

Lines Three, Four and Five

AND THE ONE-LINE OMISSION THAT LOOKS LIKE A BAD MODEL

PyTorch accumulates gradients. backward adds to whatever is already in each parameter's grad attribute rather than replacing it, which is a deliberate design decision that allows a gradient to be assembled from several pieces.

The price is that you must clear them yourself, every step, before calling backward. That is what zero_grad does, and it is the line people leave out.

The symptom is on the right, and it is the reason this gets its own slide. Without zero_grad, each step uses the sum of all gradients so far, so your effective step size grows without bound. The loss falls at first — because early on the accumulated direction is still roughly right — and then flattens out well above where it should be. It looks exactly like a model that is not flexible enough.

Then backward, which walks the graph, and step, which updates every parameter using its grad. With Adam the update is not simply gradient times learning rate — lecture six point one derives what it actually is — but the shape of the line does not change.

WITH AND WITHOUT zero_grad

Same model, same data, same seed. The only difference is one line, and the symptom looks like a capacity problem.

The One Hyperparameter That Always Matters

THREE RUNS, AND HOW TO READ THEM

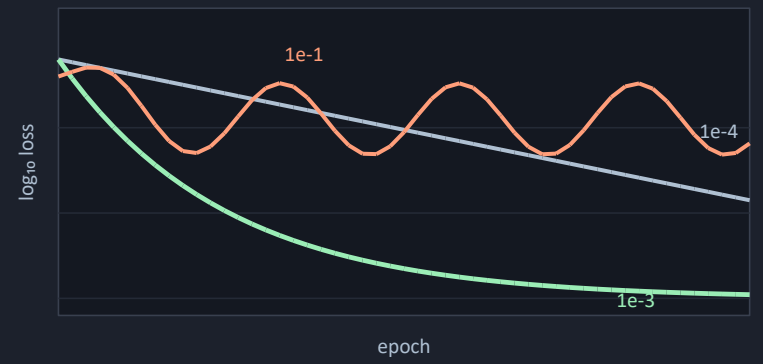
The learning rate scales the update. Too small and the run is still falling when you stop it, which wastes time but loses nothing. Too large and the loss oscillates or diverges, and the run is wasted.

One times ten to the minus three is a reasonable default for Adam on the problems in this course, and that is genuinely where you should start rather than tuning first.

Read the loss curve to decide, and read it on a logarithmic axis, for the reason L1.2 gave. Still falling steeply at the last epoch: train longer or raise the rate. Flat and low: you are done. Flat and high: check zero_grad before you touch the learning rate, because slide nineteen looks identical from here. Spiky: lower it.

Lecture six point one covers optimisers properly, including why Adam is a reasonable default at all and when it is not. Until then, one times ten to the minus three, watch the curve, and change one thing at a time.

THREE RATES, ONE PROBLEM



READING THE CURVE

still falling	train longer, or raise it
flat and high	check zero_grad first
spiky	lower it

HONESTY

Ask It Something It Was Not Shown

AND WHY THE ANSWER IS THE ARGUMENT FOR PART 2

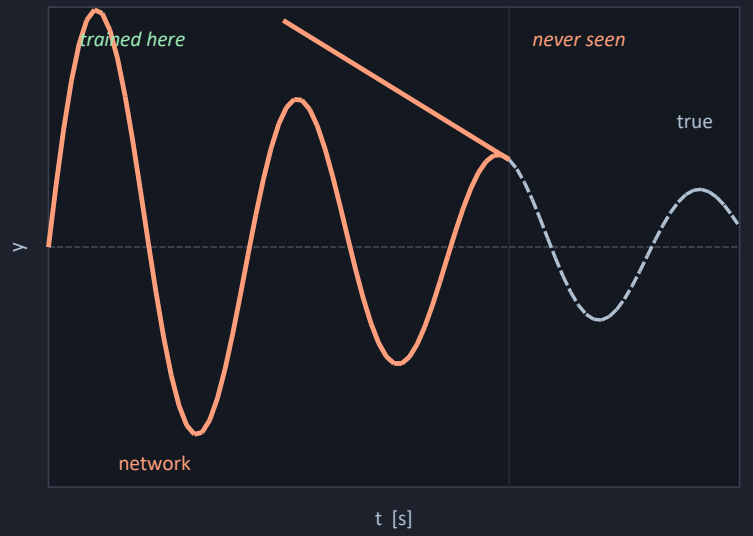
Train a network on a decaying sinusoid over one interval, then evaluate it outside that interval. Inside, it is excellent. Outside, it leaves — smoothly, confidently, and completely wrongly.

This is not a bug and it is not overfitting. The network was asked to reproduce a set of points and it did. Nothing in the loss said anything about what should happen elsewhere, so there was nothing to determine it.

You met the same result last lecture, from the other direction: the Ex03 network extrapolates a cooling curve by tens of degrees while a two-parameter fit gets it right. Lecture four point one explains the mechanism in full.

And this is precisely the argument for the second half of this course. A physics-informed network does not extrapolate better because it is a better network. It extrapolates better because the differential equation holds everywhere, including where you have no data, and the residual can be evaluated there. That is what slide thirteen bought you.

INSIDE, THEN OUTSIDE



It is confident, smooth, and wrong. Nothing in the loss ever mentioned this region.

WHERE THIS GOES

One Loop, Two Courses

THE SAME EIGHT LINES, WITH ONE LINE DIFFERENT

EX03 TO EX06 — SUPERVISED

```
for epoch in range(N):
    pred = net(t)
    loss = ((pred - y)**2).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
```

EX07 TO EX12 — PHYSICS-INFORMED

```
for epoch in range(N):
    u = net(x)
    r = d2(u, x) + k**2 * u
    loss = (r**2).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
```

WHAT CHANGED, AND WHAT DID NOT

The loop, the optimiser, zero_grad, backward and step are identical. What changed is the loss — and specifically that the second version contains a derivative of the network with respect to its input, and contains no measured data at all.

That is the whole difference between the two halves of this course, written out. Everything else in lectures seven to twelve — the physics, the geometry, the boundary conditions, the weighting between loss terms — is detail on top of those seven lines. You now have all of it except the physics.

AND THE ONE FUNCTION THAT CARRIES IT

d2 is grad of grad, with create_graph=True. It is defined in Ex02's core module and, under the same name, in every Part 2 core library.

THE EXERCISE

Ex02 — and Notebook 02 Above All

THE MOST IMPORTANT NOTEBOOK IN PART 1, AND IT SAYS SO

Notebook 00 is the environment check, including a six-line autograd smoke test. Notebook 01 is tensors: dtype, device, the column convention, a layer by hand, requires_grad and the three ways to lose the graph.

Notebook 02 is the one. Seven TODOs, and its first line says it is the most important notebook in Part 1. It builds the argument in eight steps: autograd on a scalar checked by hand, grad_outputs, an analytic function on a grid, second derivatives and create_graph, autograd against finite differences over eleven decades, a residual, the derivative of a network with respect to its input, and finally ReLU against tanh.

Notebook 03 is the training loop, fitted to a decaying sinusoid, with the zero_grad comparison and the extrapolation figure from the last two slides.

One honest note about the state of these files. The Ex02 notebooks have been checked cell by cell but not executed end to end, because torch could not be installed where they were written. The expected outputs are reasoned estimates rather than transcripts. If a number in a what-you-should-see note is out by a factor of two, that is why — the tolerances and the shapes are the parts to trust.

THE NOTEBOOKS

00	environment check — read only
01	tensors, 5 TODOs
02	autograd by hand, 7 TODOs
03	the training loop, 3 TODOs

CPU only. Notebooks 00 to 02 do no training at all and are instant; notebook 03 runs six short trainings in well under a minute.

PAUSE THE VIDEO HERE

Ex02 notebook 02, all seven TODOs. Sections 5 to 8 are the ones that pay off in week seven, and section 8 closes the loop on slide 15.

SUMMARY

Key Takeaways

1	**A tensor remembers what happened to it**	Everything else transfers from NumPy. The graph is built forwards, as a side effect of computing, and grad_fn is the evidence.
2	**Autograd is a general differentiation engine**	Not a neural-network mechanism. Exact, no step size, one backward pass regardless of how many inputs.
3	**It differentiates with respect to the INPUT**	torch.autograd.grad(u, x) gives du/dx where x is a position. That single call is the mechanism of all of Part 2.
4	**A residual is a loss with no data in it**	Move the equation to one side, square it, take the mean. That is the conceptual jump of the second course.
5	**Forward, loss, zero_grad, backward, step**	In that order. zero_grad before backward, step after. Leave it out and it looks like a model that is not flexible enough.

Next — L3.1, and the live course begins: what AI is, the three ingredients, and where a model sits on the modelling spectrum.

SOURCES

References and Further Reading

ONE TUTORIAL, ONE CHAPTER, AND ONE FORWARD POINTER

The PyTorch tutorial called A Gentle Introduction to torch.autograd, and the one on tensors that precedes it. Together about forty minutes, and they are the official version of slides three to eleven.

Prince chapter 7, on gradients and initialisation, is where the mathematics behind backward lives — the backpropagation algorithm derived properly rather than described. You do not need it today and you will need it for lecture six point one, which is where it is set.

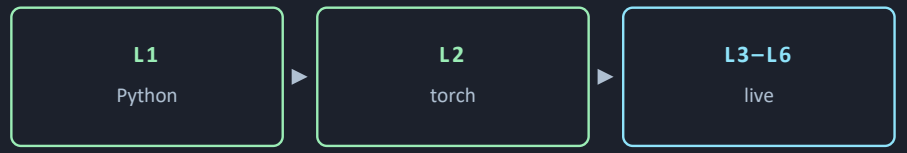
And a forward pointer rather than a reading. Lecture six point one has four bridge slides into Part 2, and the first of them is this lecture's slide twelve stated again with more machinery behind it. If slide twelve does not land today, it gets a second chance there — but the exercise is here, and the exercise is what makes it stick.

AND WHAT NOT TO DO

Do not go looking for tutorials on building neural networks yet. You have a differentiation engine and a training loop; the architecture comes in lecture four, and meeting it now out of order is how people end up copying code they cannot debug.

IN THIS ORDER

today	Ex02 notebooks 00 and 01
today	Ex02 notebook 02 — all of it
this week	the autograd tutorial
before Ex03	Ex02 notebook 03
before L6.1	UDL ch. 7
not yet	anything about architectures



The recorded lectures end here. Lecture three is live.

NEXT

L3.1 — What Is AI · and the course goes live

That is the end of the recorded material. You can write Python, you can handle arrays and their shapes, you can read a messy file and solve an equation, and you can differentiate anything with respect to anything. Everything from here is about what to differentiate, and why.

BEFORE THE FIRST LIVE LECTURE

do	Ex02 notebooks 00 to 03
above all	notebook 02, sections 5 to 8
bring	your finite-difference error plot
and	one question about it

The finite-difference plot from notebook 02 section 5 is the figure I would most like you to have produced yourself. It is the whole argument for the method in one picture, and it takes about four minutes to run.